

Node allocator benchmark

Page allocators vs. the former default pool, for `boost::root_ptr`, `std::unique_ptr` and `std::shared_ptr`

Measured 2026-09-27 on an Intel Core i7-4700HQ (4 cores / 8 threads), Linux 5.15, clang 23 -02, libstdc++ 12, glibc 2.35. Median of 5 runs; every combination runs in a fresh process. Two builds: with thread support (default) and with `BOOST_DISABLE_THREADS`. Full tables and method: `ALLOCATOR_BENCHMARK.md`.

Findings

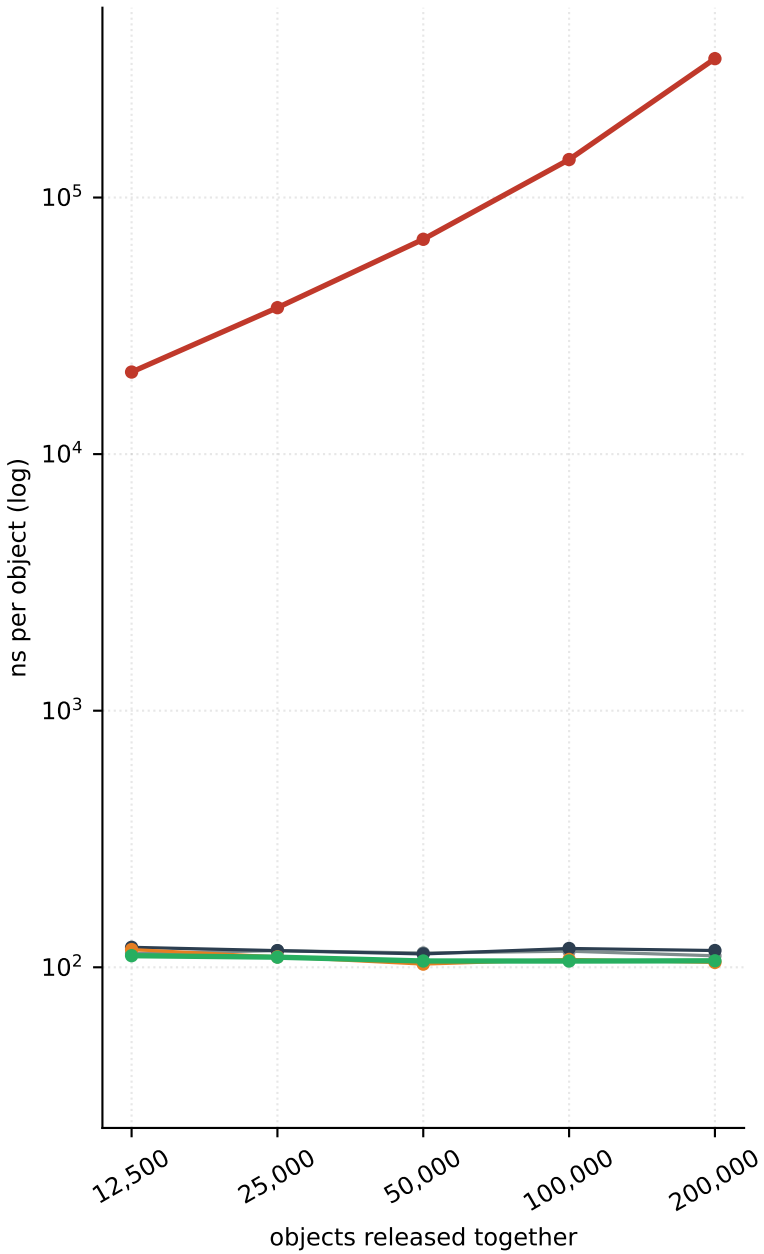
- `boost::pool_allocator`, the former default, is quadratic on release (`ordered_free`): releasing 200,000 objects costs 180-348 μ s per object; the others stay at 37-120 ns.
- `page_allocator_by_size`, now the default, is fastest or within 3% on bulk, mixed and cycle release in both builds, and uses 9-18% less memory than `boost::pool_allocator`. Churn: `std::allocator` is 8-15% faster with thread support; `fast_pool_allocator` is 1.75-2x faster for `unique_ptr/shared_ptr` without it.
- Thread support is `root_ptr`'s largest cost: without it, churn drops 57.8 -> 9.5 ns (6.1x), cycles 122 -> 44 ns per node. `root_ptr` then matches `shared_ptr` on churn.
- `page_allocator_by_type` costs 2.7-4.2x the time and 1.6-1.9 MB more memory than `by_size` with many sparse types: each type fills a 64 KiB page of its own.
- With four threads, `std::allocator` is 28-36x faster than any pool for `unique_ptr` and `shared_ptr`; for `root_ptr` the global mutex narrows the gap to 1.5x.

Pages (with thread support unless noted)

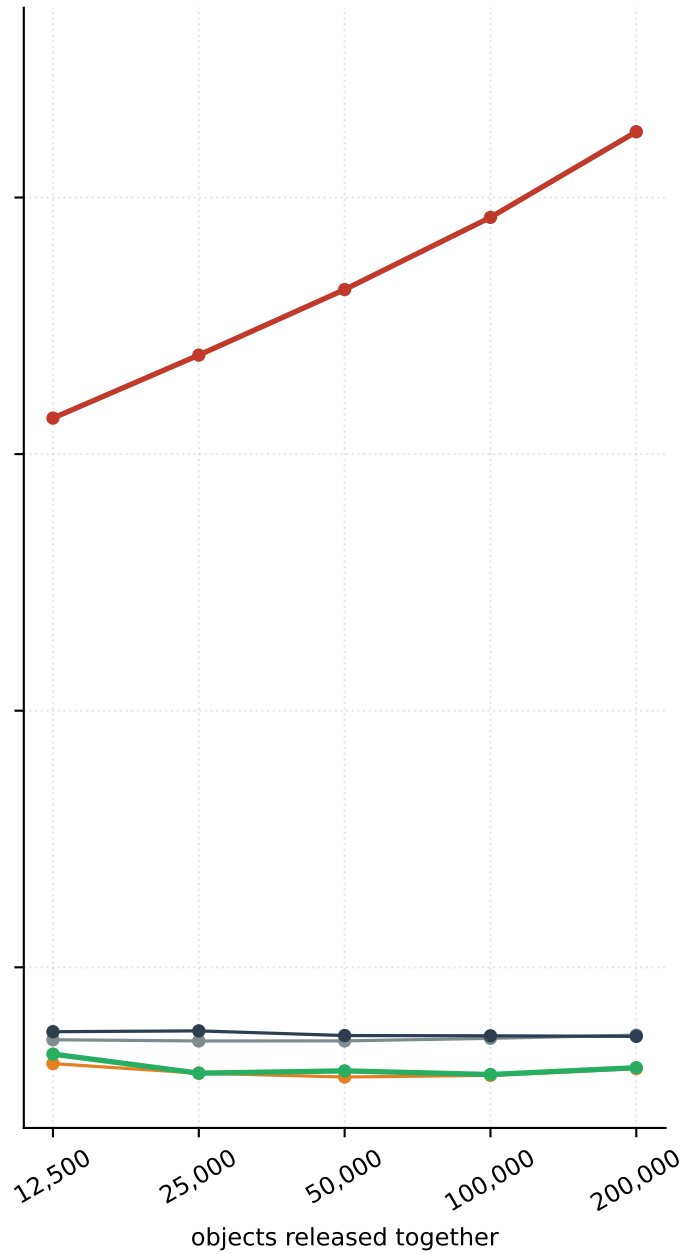
- | | | |
|-----------------------------|---------------------|--------------------------|
| 2 Bulk release scaling | 3 Time per scenario | 4 Resident memory |
| 5 Pointer types compared | 6 Threads | 7 Without thread support |
| 8 What thread support costs | | |

Bulk release scaling: boost::pool_allocator grows with the number of objects, the others stay flat

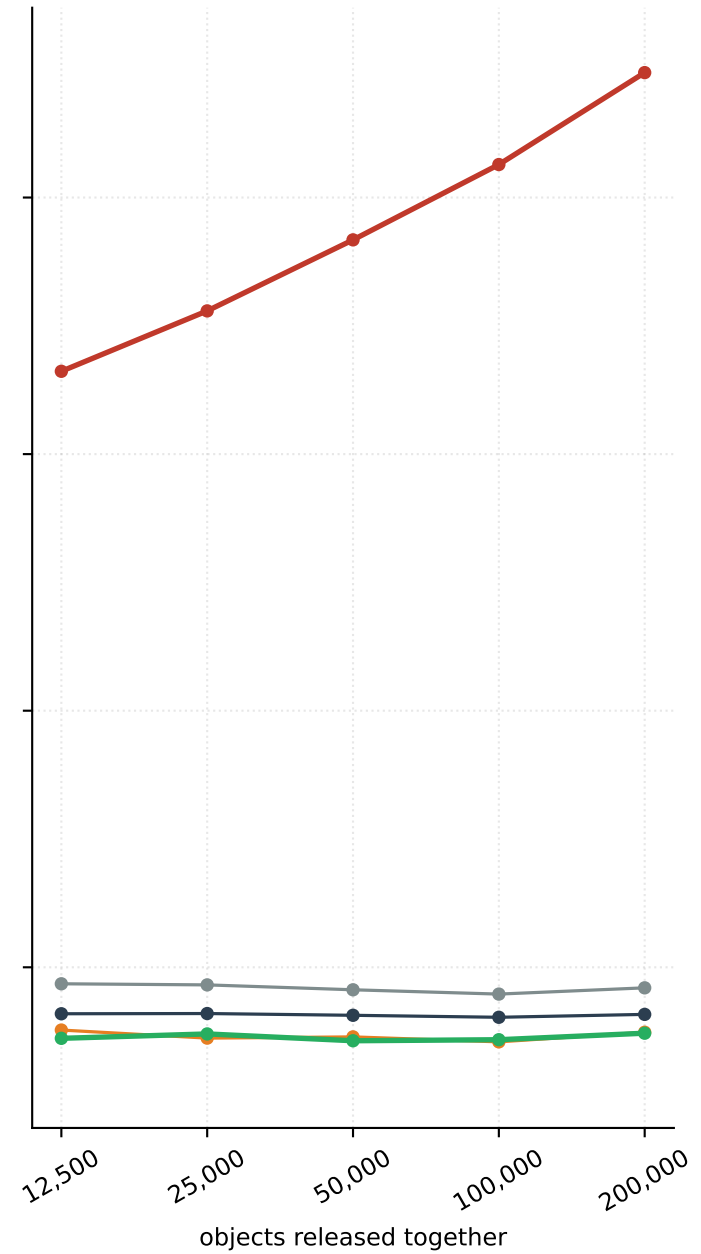
boost::root_ptr



std::unique_ptr

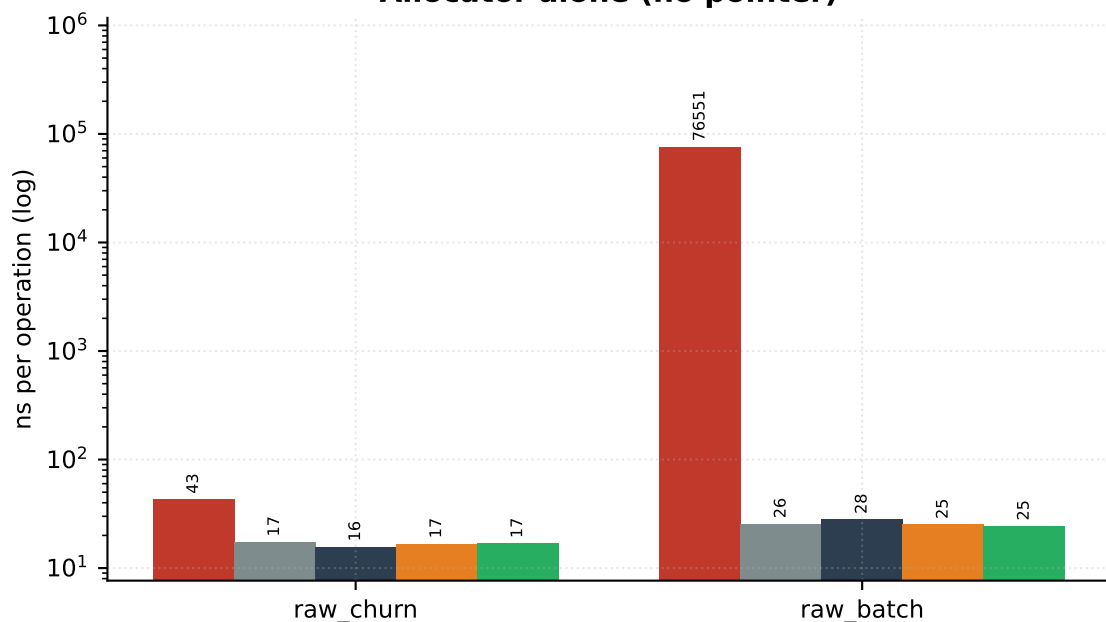


std::shared_ptr

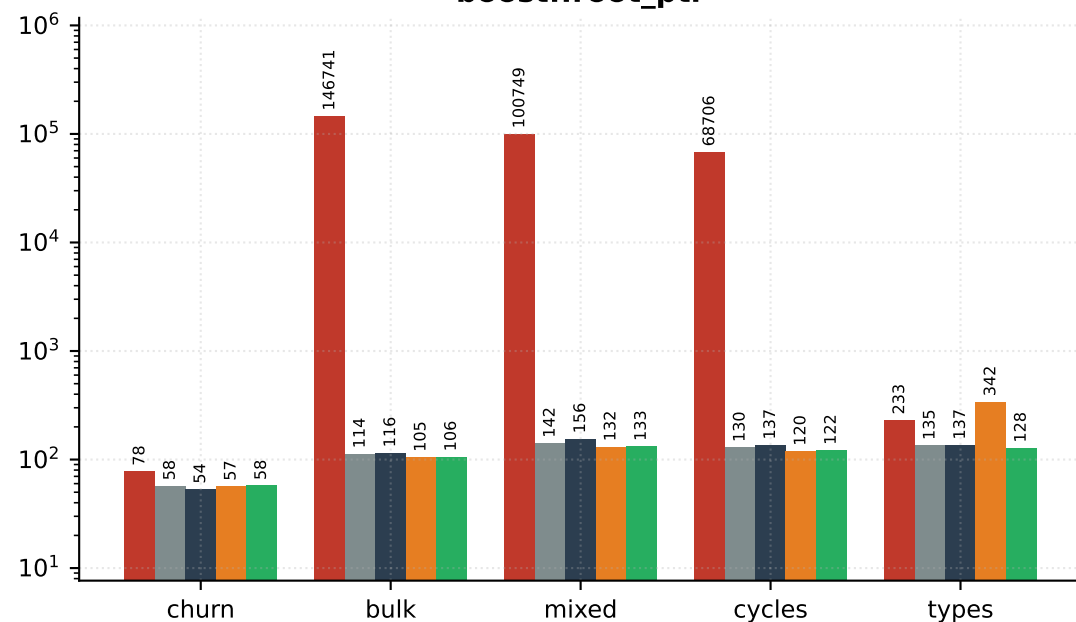


Time per operation, single-threaded scenarios, built with thread support (lower is better)

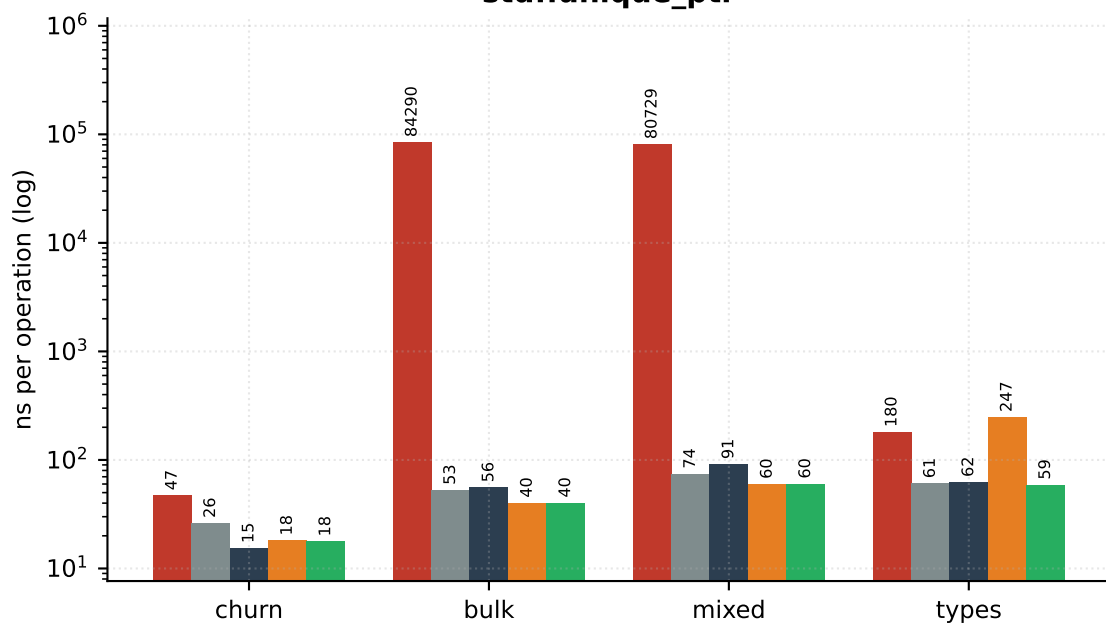
Allocator alone (no pointer)



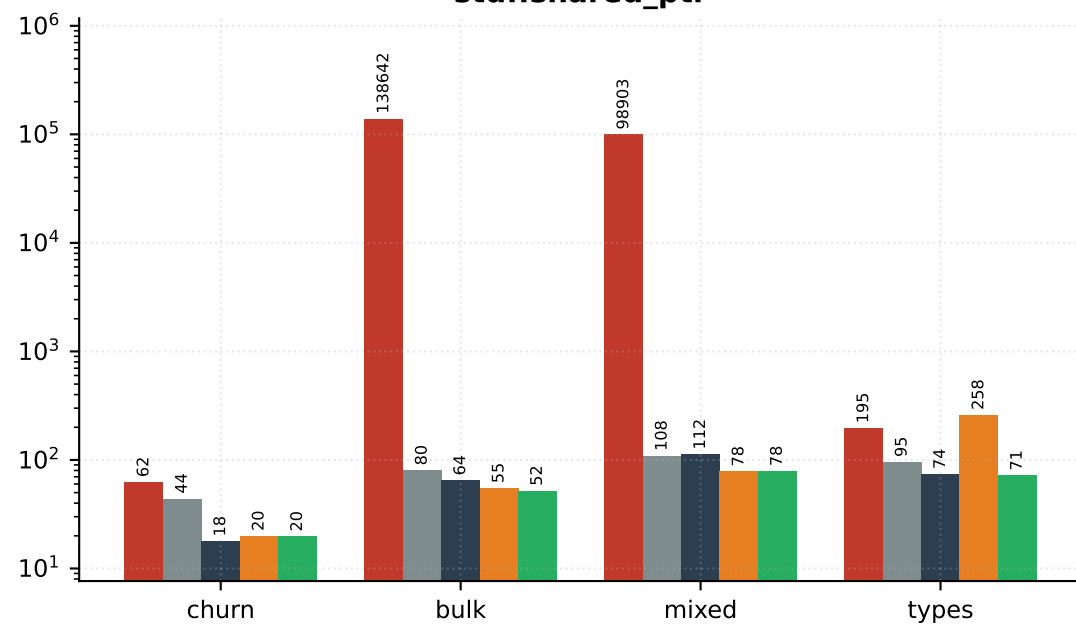
boost::root_ptr



std::unique_ptr



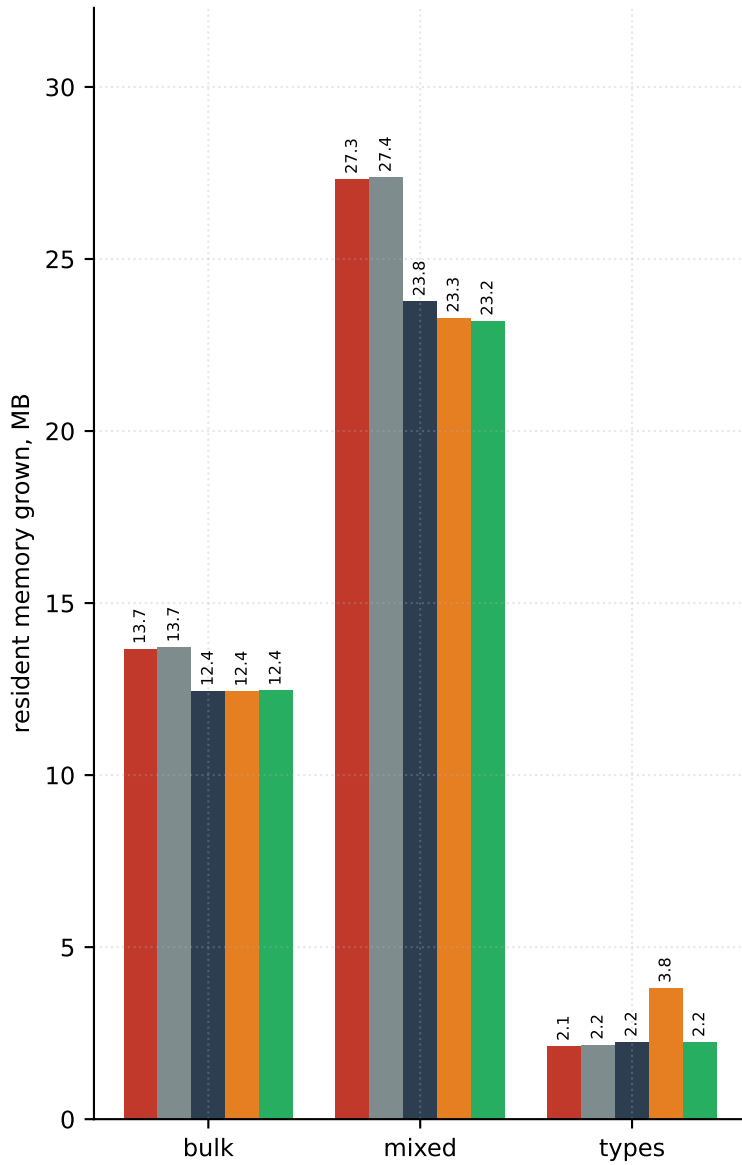
std::shared_ptr



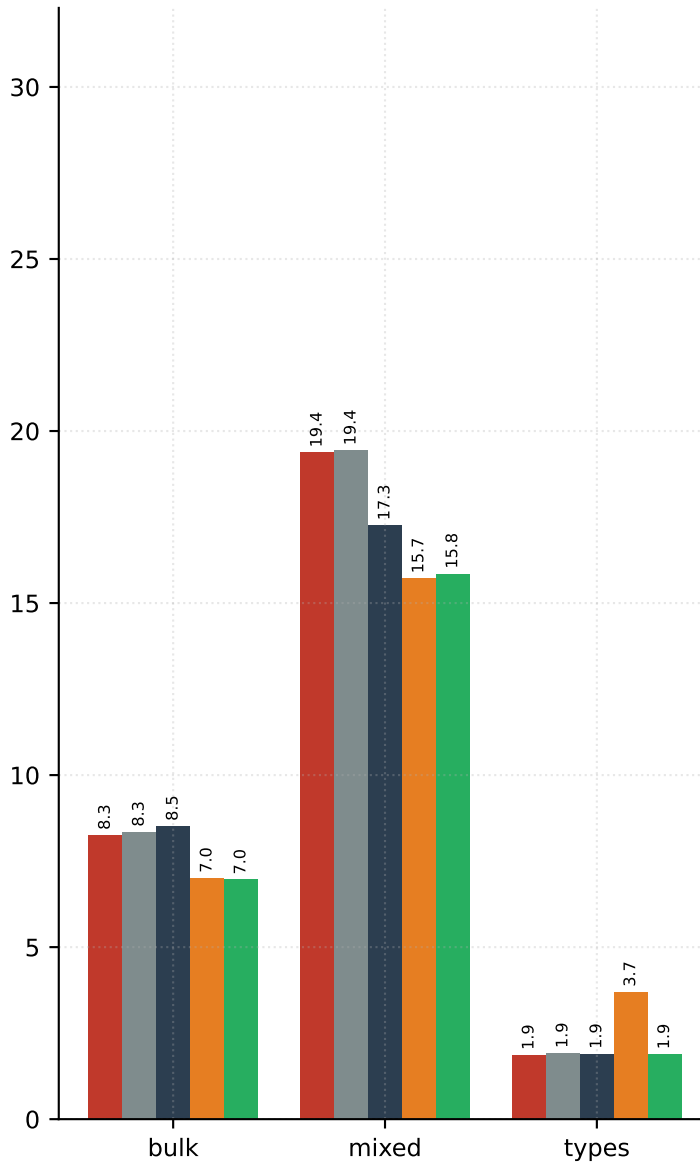
boost::pool_allocator (former default) boost::fast_pool_allocator std::allocator page_allocator_by_type page_allocator_by_size (default)

Resident memory while the objects are live (lower is better)
bulk: 100,000 x 48 B mixed: 50,000 each of 16, 48 and 200 B types: 32 types x 100 objects

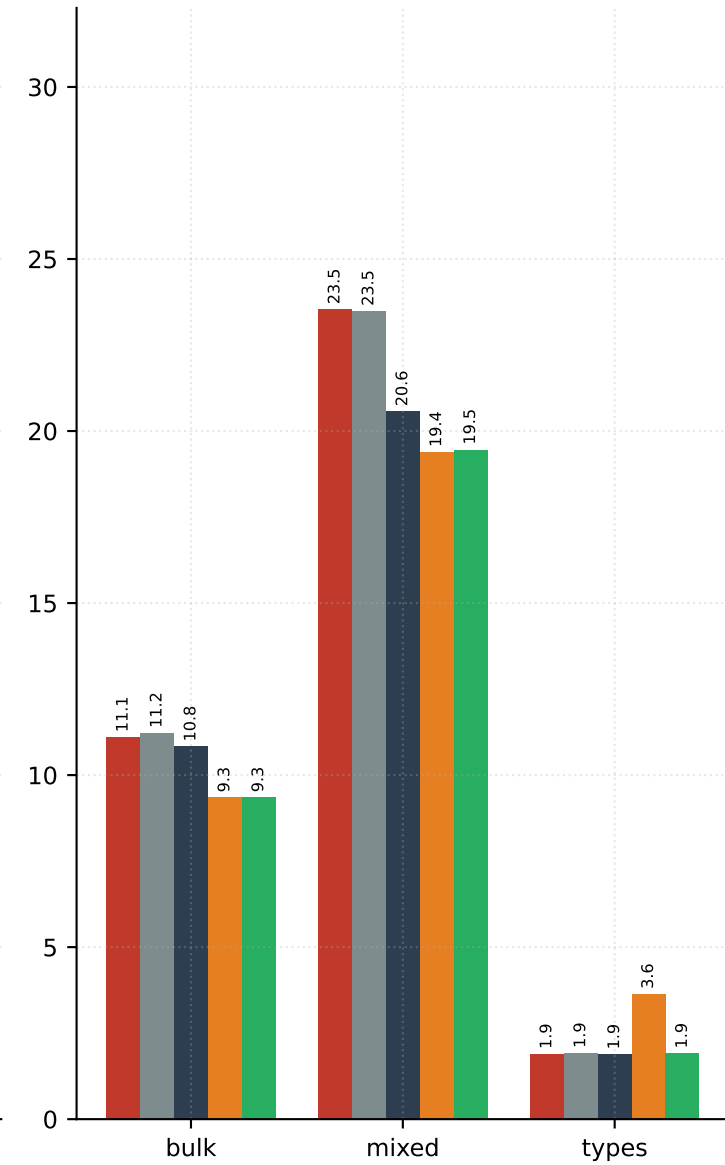
boost::root_ptr



std::unique_ptr



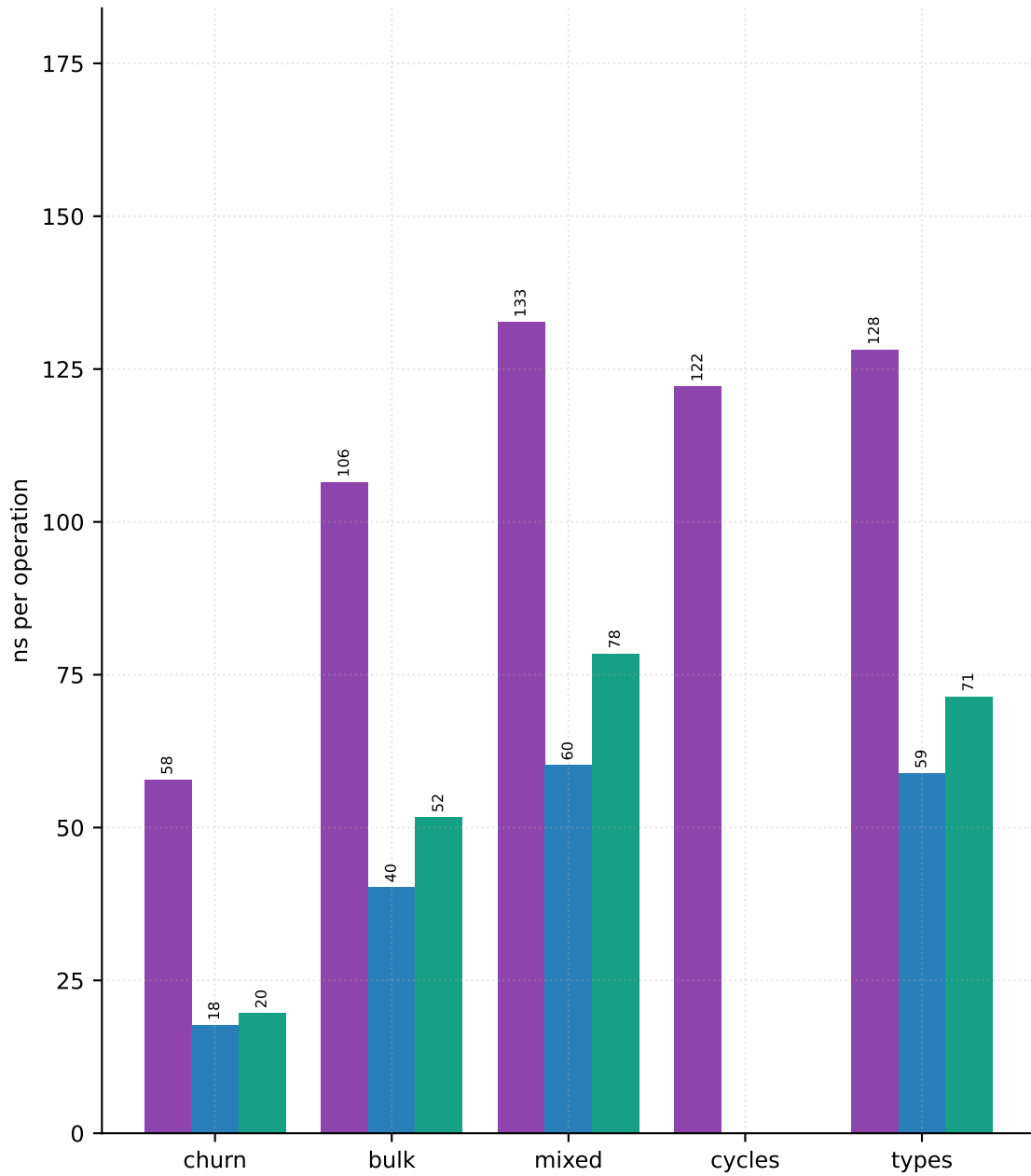
std::shared_ptr



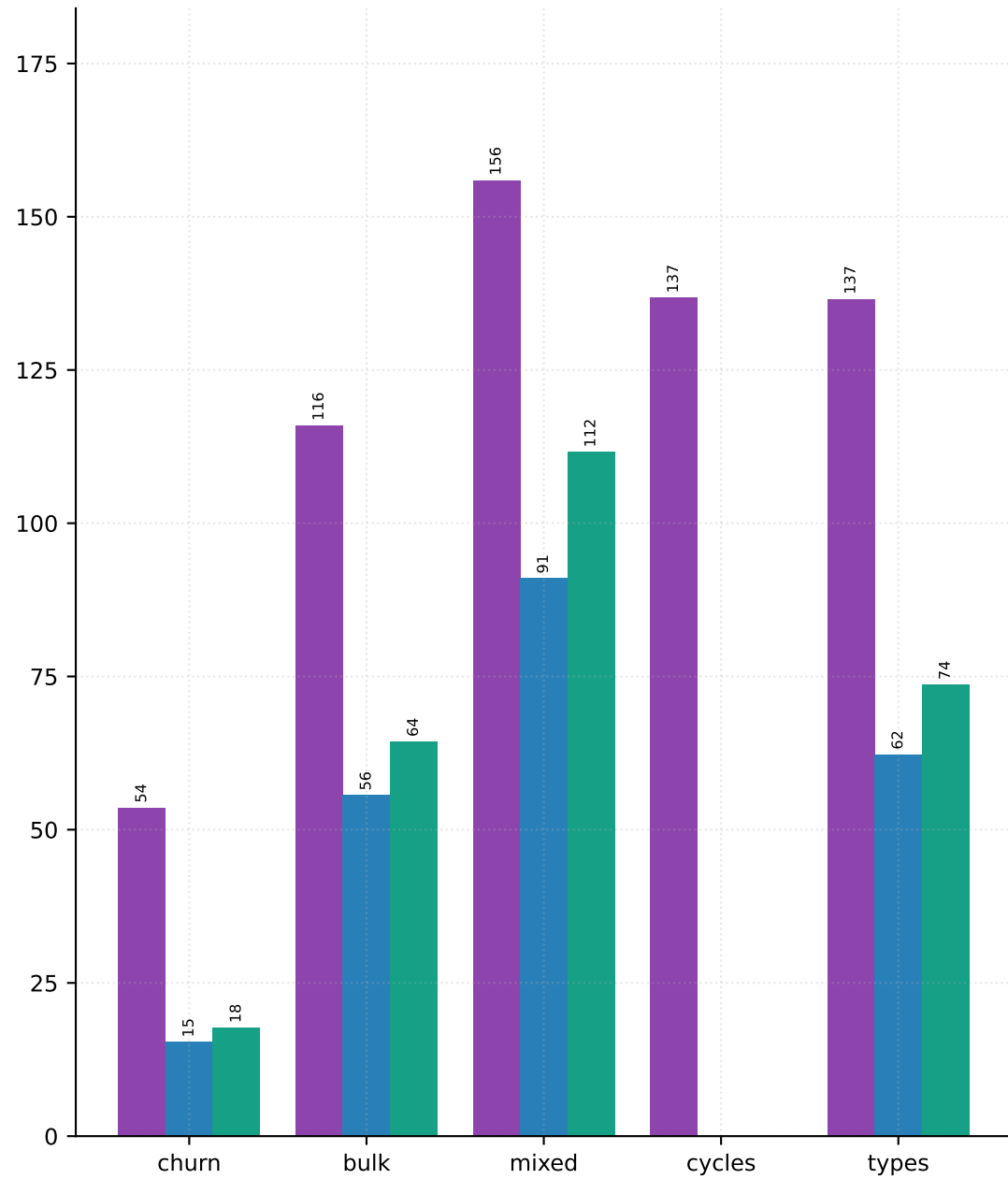
boost::pool_allocator (former default) boost::fast_pool_allocator std::allocator page_allocator_by_type page_allocator_by_size (default)

Pointer types compared (cycles: root_ptr only - a unique_ptr cannot form one, a shared_ptr cycle is never freed)

with page_allocator_by_size (default)



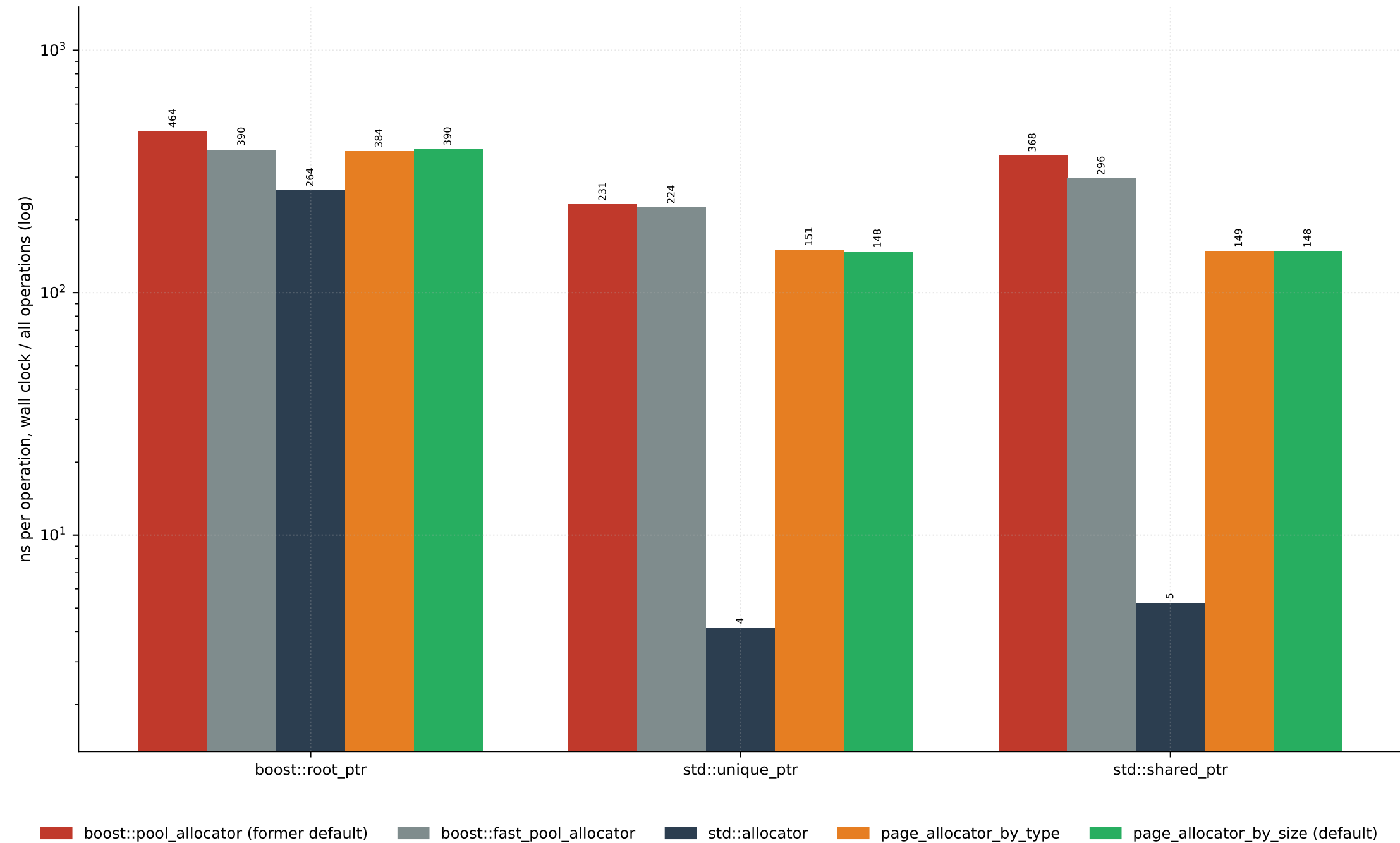
with std::allocator



boost::root_ptr std::unique_ptr std::shared_ptr

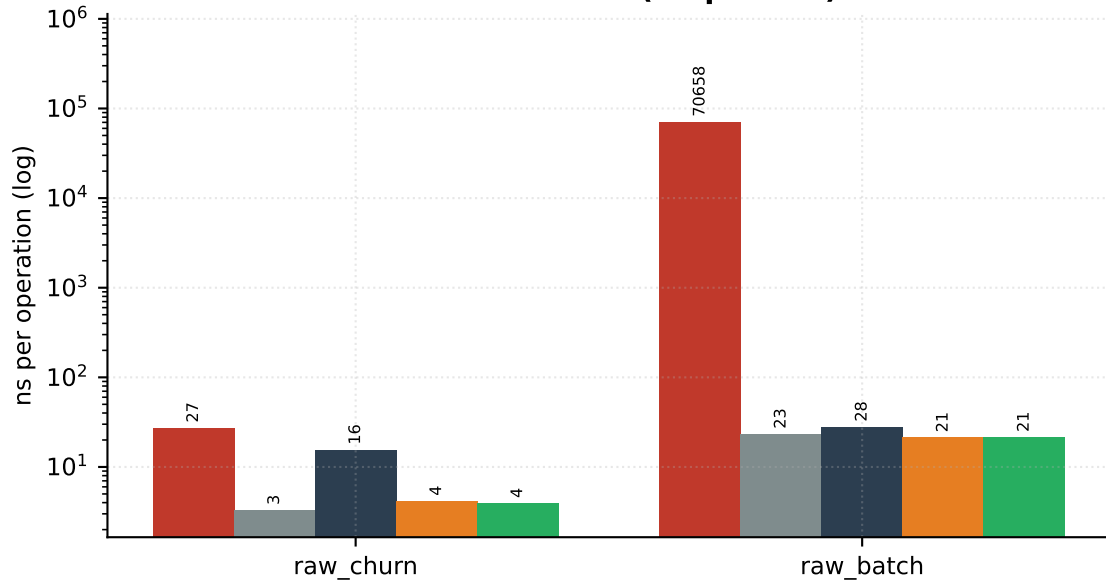
4 threads, each creating and dropping 250,000 objects (lower is better)

glibc's per-thread caches take no lock; each page pool has one mutex; every root_ptr operation takes a global mutex

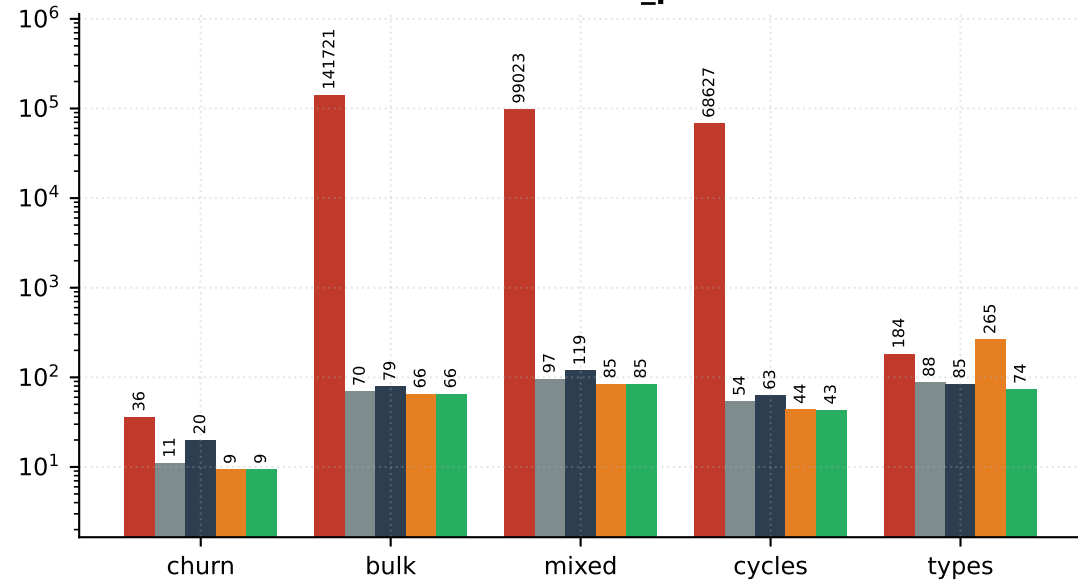


**Time per operation, built with BOOST_DISABLE_THREADS (lower is better)
no root_ptr mutex, plain reference counts, no pool mutexes**

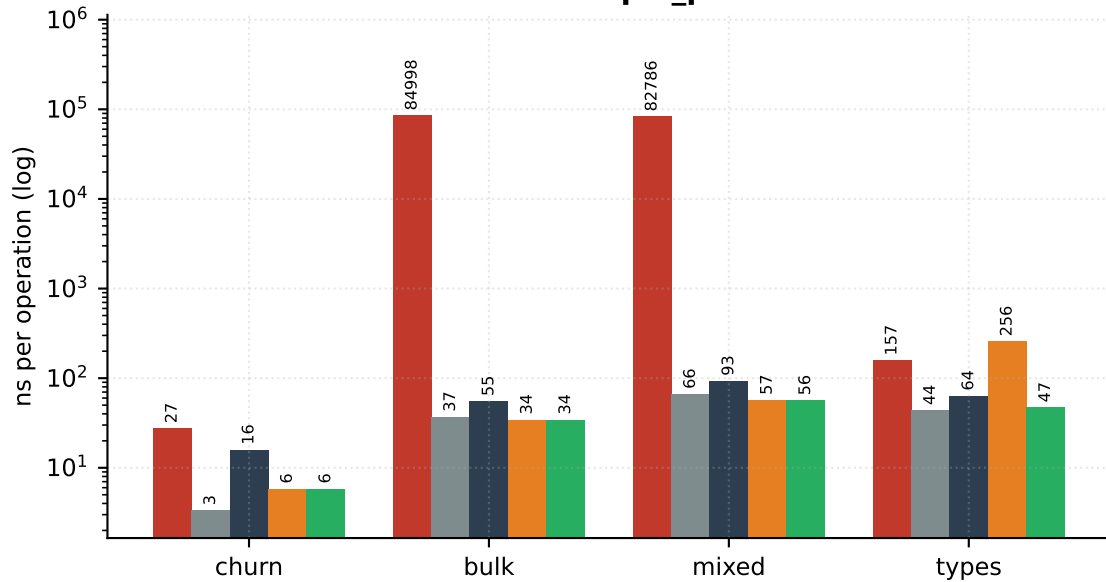
Allocator alone (no pointer)



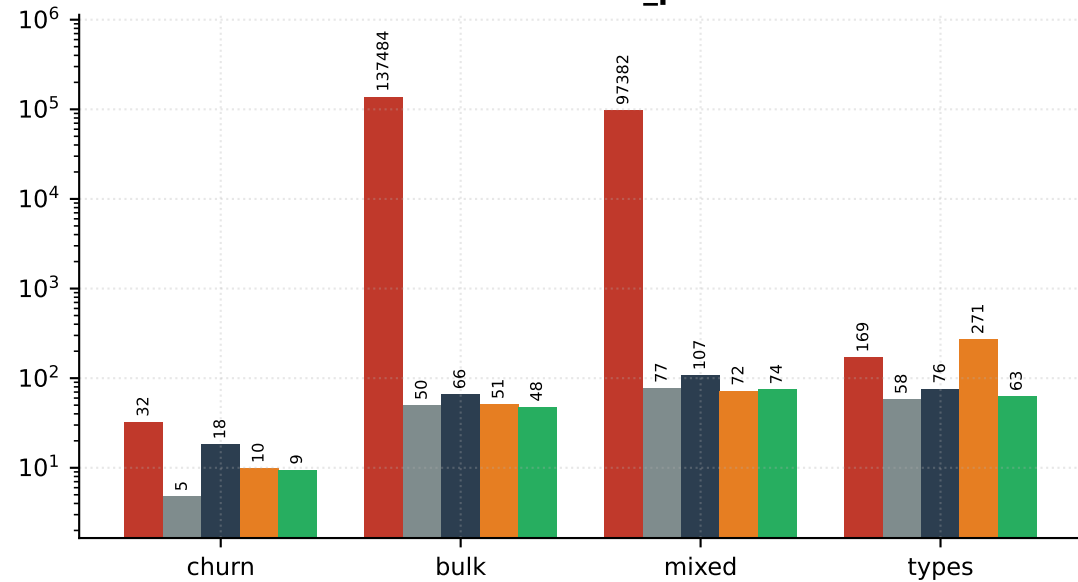
boost::root_ptr



std::unique_ptr



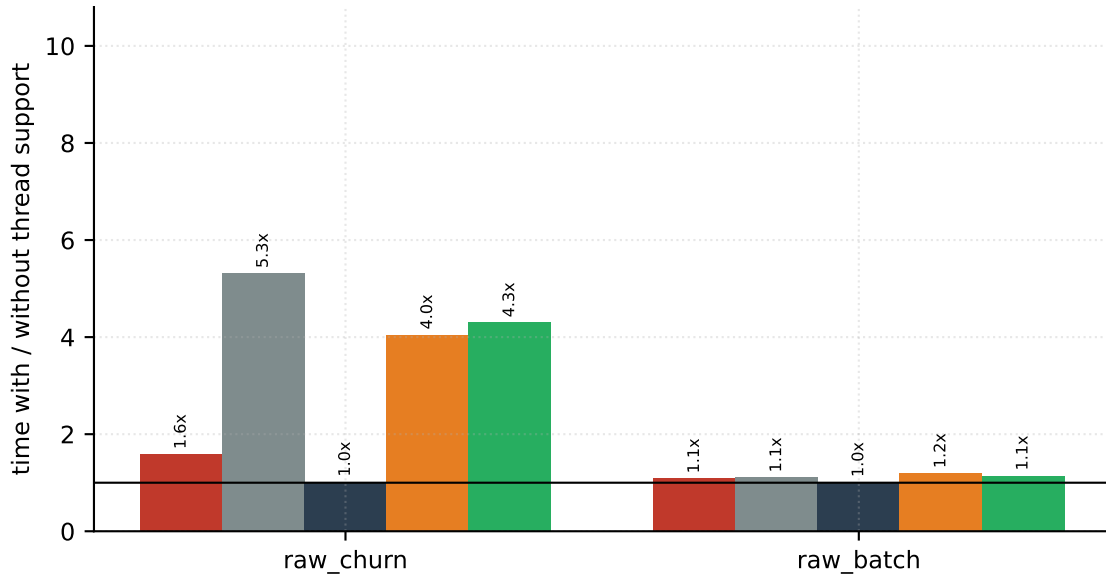
std::shared_ptr



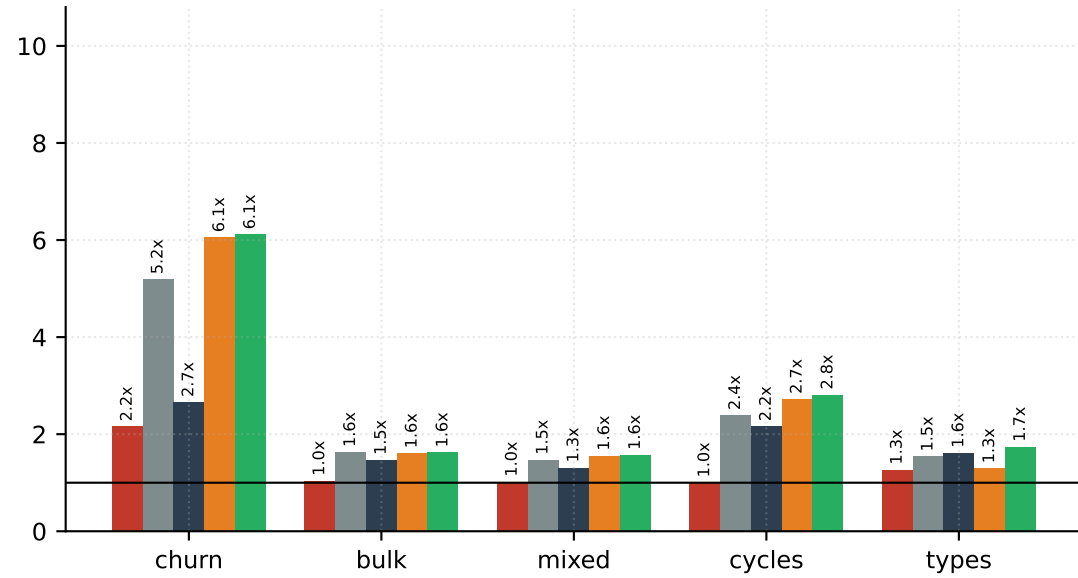
boost::pool_allocator (former default) boost::fast_pool_allocator std::allocator page_allocator_by_type page_allocator_by_size (default)

What thread support costs: time with thread support divided by time with BOOST_DISABLE_THREADS (1.0x = no cost; higher = thread support is more expensive)

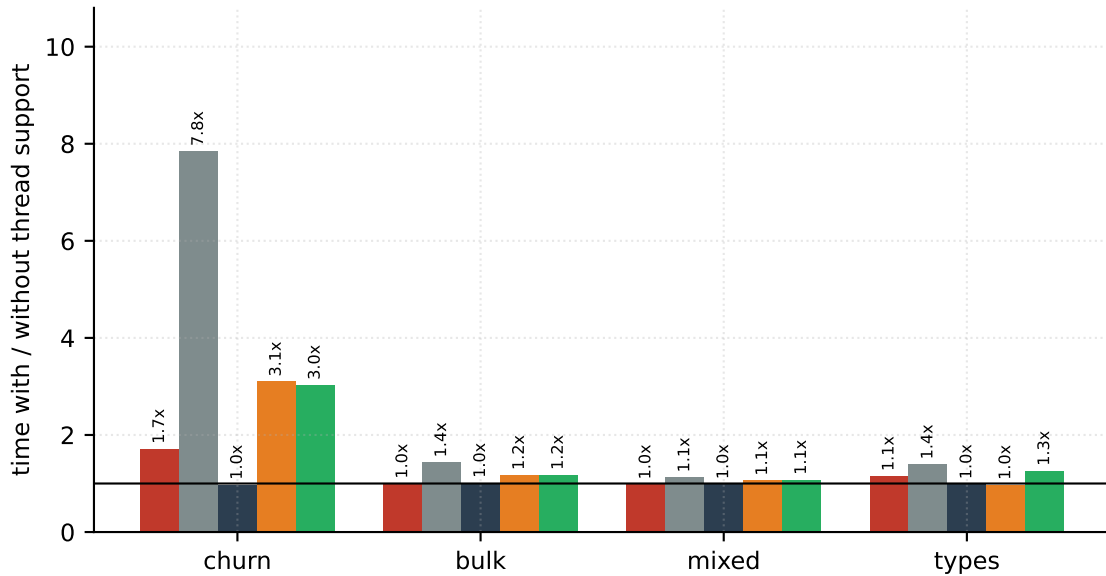
Allocator alone (no pointer)



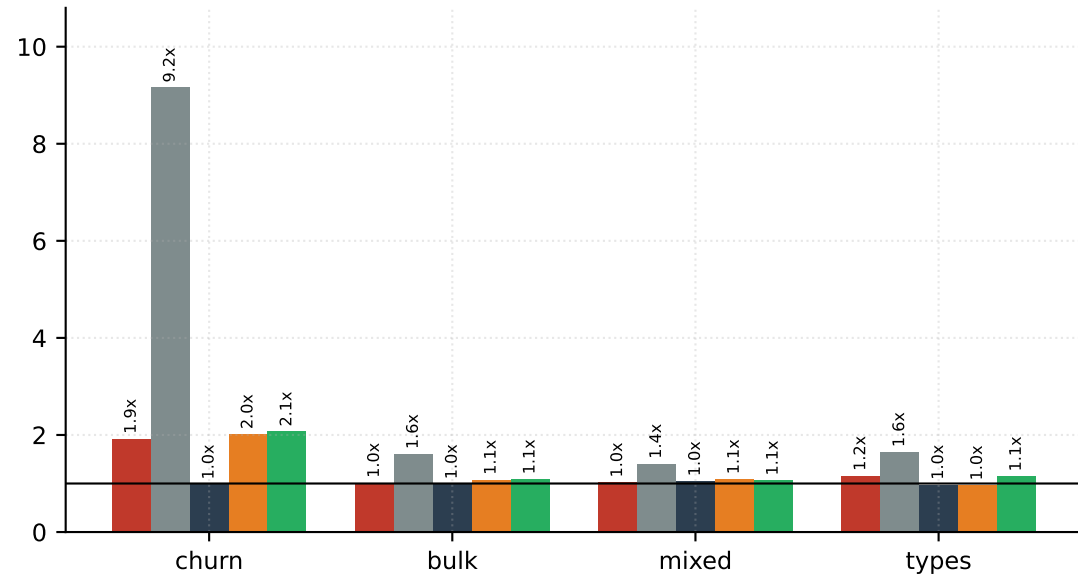
boost::root_ptr



std::unique_ptr



std::shared_ptr



boost::pool_allocator (former default) boost::fast_pool_allocator std::allocator page_allocator_by_type page_allocator_by_size (default)